

# Context Engineering（上下文工程）

面向AI智能体的上下文工程策略：写入、筛选、压缩、隔离

原文作者：Lance Martin · LangChain

团队 | 原文链接：[langchain.com/blog/context-engineering-for-agents](https://langchain.com/blog/context-engineering-for-agents)

中文版 · Cola 译制 · 2026-06-26

## 太长不看（TL;DR）

智能体（Agent）需要上下文来完成任务。上下文工程（Context Engineering），就是在智能体轨迹的每一步，向上下文窗口中填入恰如其分的信息的一门艺术与科学。本文通过回顾一批代表性的智能体产品与论文，拆解上下文工程的四大核心策略——写入（Write）、筛选（Select）、压缩（Compress）、隔离（Isolate），并说明 LangGraph 是如何原生支持这四种策略的。

相关视频讲解可在LangChain官方频道观看。

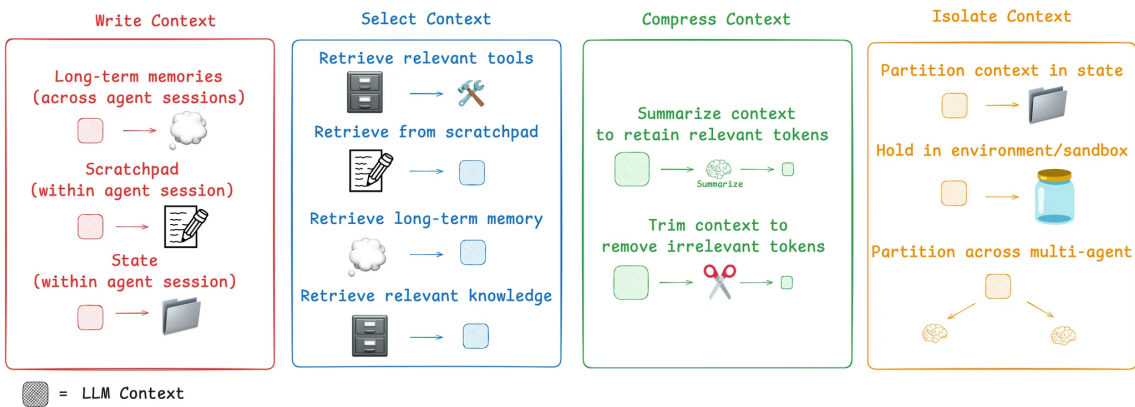


图1 上下文工程四大策略总览

## 一、什么是上下文工程

用 Andrej Karpathy 的比喻来说，大语言模型（LLM）像是一种新型操作系统：LLM 本身是 CPU，它的上下文窗口就是 RAM——也就是模型的工作内存。和 RAM 一样，上下文窗口能容纳的信息是有限的，需要容纳来自不同来源的上下文。而正如操作系统要精心挑选什么东西能装入 CPU 的 RAM，我们同样可以把“上下文工程”理解为扮演这一角色的工作。

Karpathy 对此有一句精准的概括：

“上下文工程……是在每一步，都把下一步所需的、恰好正确的信息填入上下文窗口的精妙艺术与科学。”

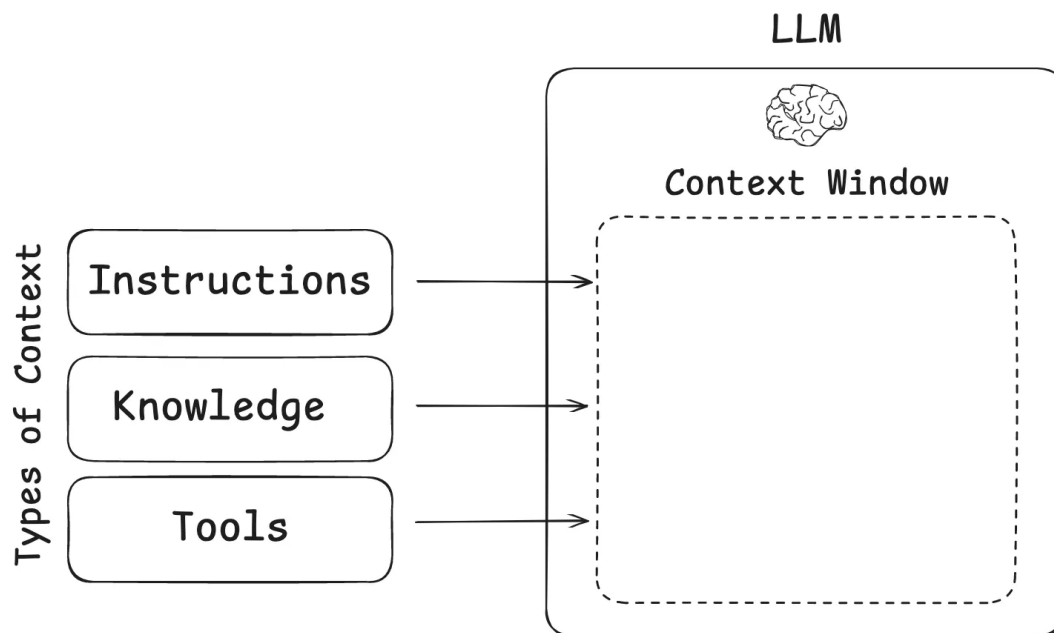


图2 LLM应用中常见的上下文类型

那么，构建 LLM 应用时我们需要管理哪些类型的上下文呢？上下文工程作为一个总括性概念，覆盖了以下几类典型的上下文：

- 指令（Instructions）：提示词（prompts）、记忆、few-shot 示例、工具描述等；
- 知识（Knowledge）：事实、用户画像、长期记忆等；
- 工具（Tools）：工具调用（tool call）返回的反馈。

## 二、面向智能体的上下文工程

2025年以来，随着 LLM 在推理和工具调用方面的能力显著提升，人们对智能体的兴趣急剧增长。智能体在执行任务时，会交错调用 LLM 与工具，通常用于处理长时间运行的任务——LLM 发出决策，调用工具得到反馈，再根据反馈决定下一步行动，如此反复。

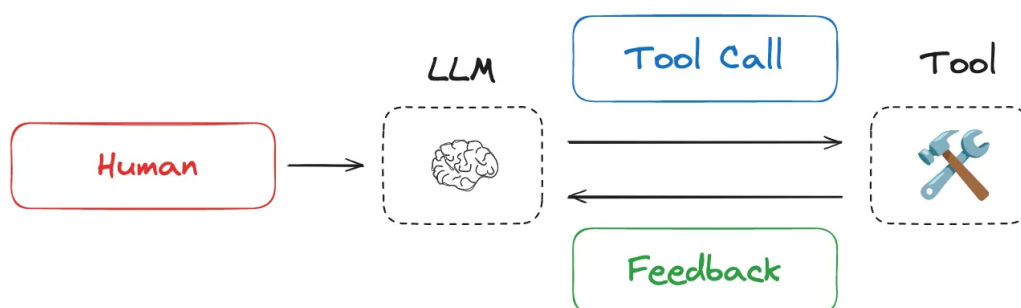


图3 智能体在LLM调用与工具调用之间交替，用工具反馈决定下一步

然而，长时任务与不断累积的工具反馈，意味着智能体往往会消耗大量 token。这会带来一系列问题：超出上下文窗口上限、成本与延迟急剧膨胀、甚至导致智能体表现退化。Drew Breunig 曾清晰地总结了“上下文过长”会导致的四类典型问题，被称为 Context Rot（上下文腐烂）：

- 上下文污染（Context Poisoning）：幻觉内容进入上下文，污染后续推理；
- 上下文分心（Context Distraction）：上下文过载，压倒模型的训练先验；
- 上下文混淆（Context Confusion）：无关上下文误导模型回答；
- 上下文冲突（Context Clash）：上下文不同部分的信息彼此矛盾。

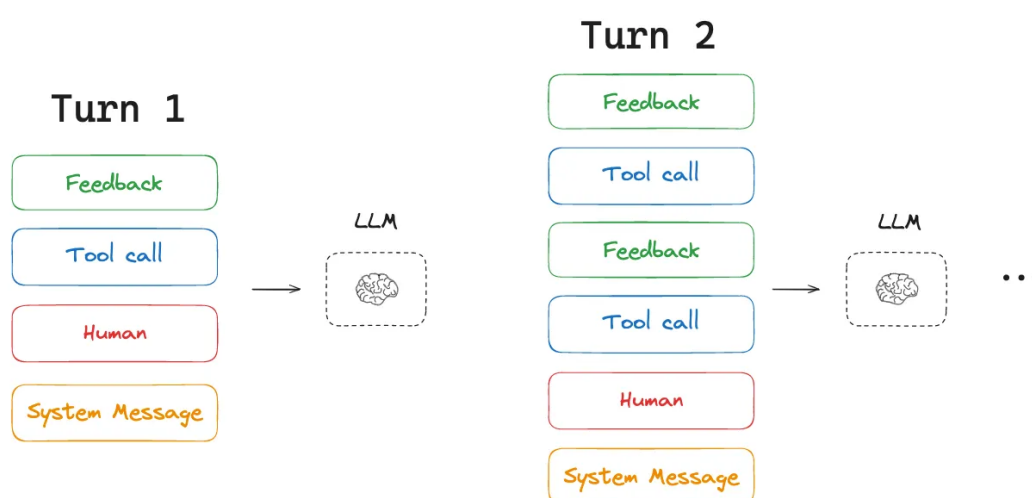


图4 多轮工具调用累积的上下文

正是因为这些问题，Cognition（Devin 的开发团队）直接喊出了这样的观点：

“‘上下文工程’……实际上是构建 AI 智能体的工程师的头号工作。”

Anthropic 也在工程博客中明确指出：

“智能体往往要持续数百轮对话，这要求我们必须有谨慎的上下文管理策略。”

那么，今天的业界是如何应对这一挑战的？我们把常见的智能体上下文工程策略归纳为四大类——写入、筛选、压缩、隔离，并结合当下主流智能体产品和论文中的实例逐一说明。最后再介绍 LangGraph 是如何为这四大策略提供原生支持的。

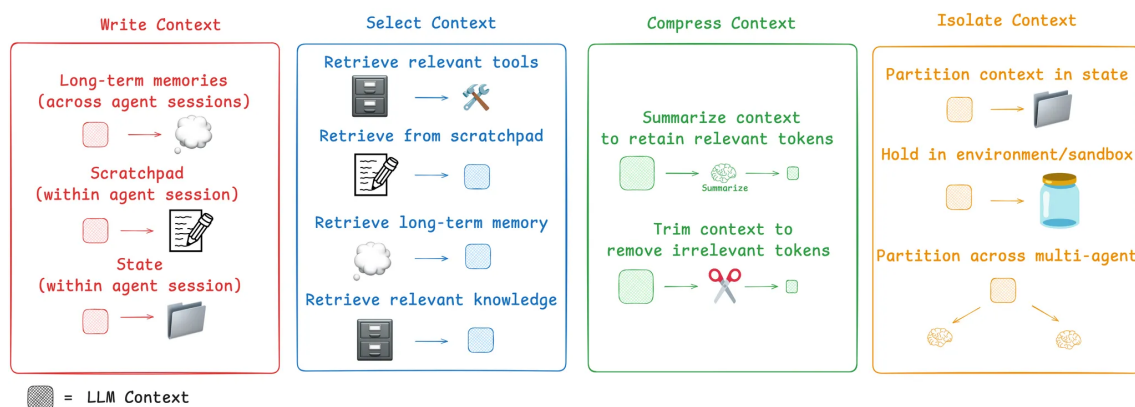


图5 上下文工程四大策略总览（重复出现，强调四类策略）

## 三、写入上下文 (Write Context)

写入上下文，就是把信息保存在上下文窗口之外，以帮助智能体完成任务。

### 3.1 草稿本 (Scratchpads)

人在解决问题时会记笔记、保存要点，方便后续相关任务使用。智能体也在获得这种能力！通过“草稿本 (scratchpad)”做笔记，是智能体在任务执行过程中持久化信息的典型做法。核心思路是：把信息保存到上下文窗口之外，让智能体在需要时可以读取。Anthropic 的多智能体研究助手就是一个非常清晰的例子：

“LeadResearcher（牵头研究员）会先构思研究路径，把计划保存到 Memory 中以持久化上下文——因为一旦上下文窗口超过 20 万 token 就会被截断，而保留计划至关重要。”

草稿本可以有多种实现方式：它可以是一个“写文件”的工具调用，也可以是运行时状态对象中的一个持久化字段。无论哪种方式，草稿本都让智能体能够把有用的信息保存下来，以支撑任务完成。

### 3.2 记忆 (Memories)

草稿本主要帮助智能体在单次会话（或线程）内解决任务，但很多时候，智能体还需要在跨越多轮会话时记住事情。Reflexion 提出了在每轮智能体行动后进行“反思 (reflection)”并复用这些自生成记忆的思路；Generative Agents 则会从智能体过去的反馈集合中周期性地合成记忆。

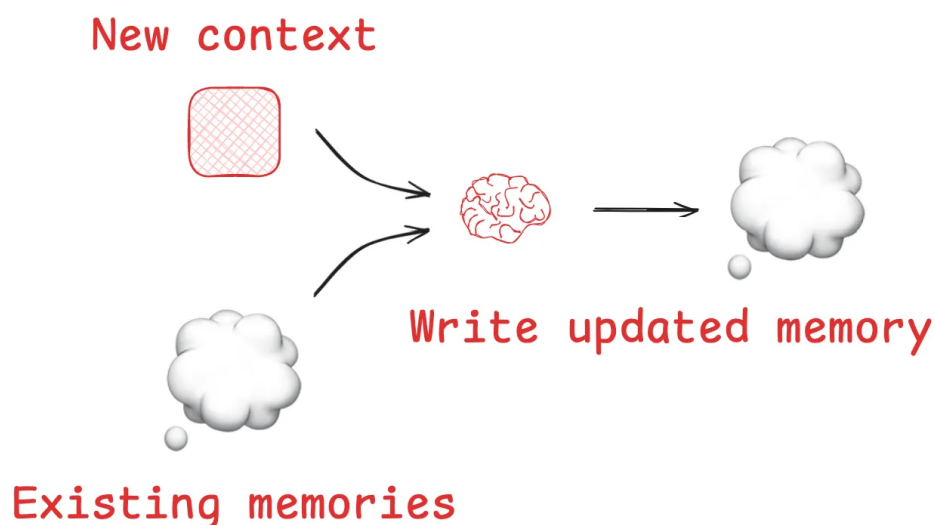


图6 可以用LLM来更新或创建记忆

这些思路已经进入了 ChatGPT、Cursor、Windsurf 等主流产品——它们都提供了基于用户-智能体交互、自动生成跨会话长期记忆的机制。

## 四、筛选上下文 (Select Context)

筛选上下文，就是把信息拉取到上下文窗口之中，以帮助智能体完成任务。

### 4.1 草稿本读取

从草稿本中筛选上下文的机制，取决于草稿本的实现方式。如果草稿本以工具形式存在，智能体就可以直接通过工具调用来读取；如果草稿本是智能体运行时状态 (runtime state) 的一部分，那么开发者可以在每一步精细选择要暴露给智能体的状态片段，从而实现对后续轮次暴露给 LLM 的上下文的细粒度控制。

### 4.2 记忆筛选

如果智能体具备保存记忆的能力，那它同样需要能够在任务执行时筛选出与当前任务相关的记忆。这在几个场景下都很重要：智能体可能会筛选 few-shot 示例（情景记忆）来获得期望行为的示范、筛选指令（程序性记忆）来引导行为、或筛选事实（语义记忆）来获得任务相关的上下文。

| Memory Type | What is Stored | Human Example              | Agent Example       |
|-------------|----------------|----------------------------|---------------------|
| Semantic    | Facts          | Things I learned in school | Facts about a user  |
| Episodic    | Experiences    | Things I did               | Past agent actions  |
| Procedural  | Instructions   | Instincts or motor skills  | Agent system prompt |

图7 从记忆库中筛选相关上下文

这里的一大挑战是：如何确保选中的记忆真的是相关的。一些主流智能体使用一组固定的、始终会被拉取到上下文的文件，这相对简单。例如，很多代码智能体会用特定文件保存指令（“程序性”记忆），某些情况下也会保存示例（“情景”记忆）：Claude Code 使用 `CLAUDE.md`，Cursor 和 Windsurf 使用 `rules` 文件。

但如果智能体保存的是更大规模的事实或关系集合（如语义记忆），筛选就变得更困难。ChatGPT 就是一个典型案例：它从海量的用户个性化记忆中存储和筛选相关内容。

嵌入向量（embeddings）和/或知识图谱通常被用来辅助记忆索引与筛选，但即便如此，记忆筛选依然是个难题。在 `AI Engineer World's Fair` 上，Simon Willison 分享了一个筛选“翻车”的例子：ChatGPT 从他的记忆中错误地读出了地理位置，并在他请求生成图片时擅自把地点注入了画面——这类意料之外或不符合用户预期的记忆召回，会让一些用户觉得上下文窗口“不再属于自己”。

### 4.3 工具筛选

智能体需要使用工具，但如果给它提供太多工具，它就会“过载”——这通常是因为工具描述彼此重叠，导致模型在该调用哪个工具上产生混淆。一种思路是对工具描述应用RAG（检索增强生成），每次只拉取与当前任务最相关的工具。近期一些论文表明，这种做法可以让工具选择准确率提升约3倍。

### 4.4 知识筛选（RAG）

RAG 是一个内涵丰富的话题，也是上下文工程的核心挑战之一。代码智能体是 RAG 大规模生产落地的最佳例子之一。Windsurf 的 Varun 对其中的挑战有一段很到位的描述：

“代码索引  $\neq$  上下文检索……我们在做索引和嵌入搜索，用 AST 解析代码、按语义边界进行 chunking……但随着代码库规模增大，嵌入搜索作为一种检索启发式方法会变得不可靠……我们必须结合 grep / 文件搜索、基于知识图谱的检索，以及一个重排序步骤，按相关性对上下文排序。”

## 五、压缩上下文（Compress Context）

压缩上下文，就是只保留完成任务所必需的 token。



## 5.1 上下文摘要 (Summarization)

智能体交互可能横跨数百轮，并伴随大量高 token 消耗的工具调用。摘要是对这些问题的常见手段。如果你用过 Claude Code，你一定见过它的实际表现：当上下文窗口使用率超过 95% 时，Claude Code 会运行 “auto-compact（自动压缩）”，把整段用户-智能体交互轨迹做摘要。对智能体轨迹的这种压缩，可以采用递归摘要、分层摘要等多种策略。

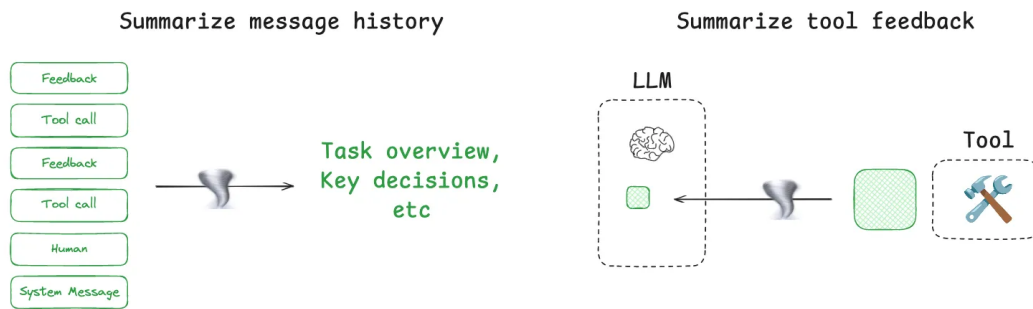


图8 摘要可以应用在智能体流程的多个位置

在智能体设计的特定位置加入摘要也很有用：例如，可以用它来对某些工具调用的结果做后处理（尤其是高 token 消耗的搜索工具）。又如，Cognition 提到会在“智能体-智能体边界”上做摘要，以在知识交接时减少 token 数。当然，摘要的一个挑战在于：必须保证关键事件和决策不被丢失。Cognition 为此使用了一个微调模型来做摘要，可见这一步可以有多大的工作量。

## 5.2 上下文修剪 (Trimming)

摘要通常依赖 LLM 来提炼最相关的上下文片段；而修剪 (trimming) 则更多是通过过滤或者——按 Drew Breunig 的说法——“剪枝 (pruning)”来完成上下文缩减。这可以使用硬编码启发式规则，例如从消息列表中移除较旧的消息。Drew 还提到 Provence，一个专门用于问答场景的、经过训练的上下文剪枝器。

# 六、隔离上下文 (Isolate Context)

隔离上下文，就是把上下文拆分，以帮助智能体更好地完成任务。

## 6.1 多智能体 (Multi-agent)

隔离上下文最流行的做法之一，就是把上下文拆分到多个子智能体之间。OpenAI Swarm 库的一个动机就是关注点分离：让一个智能体团队各自负责特定的子任务，每个智能体只拥有自己的工具集、指令和独立的上下文窗口。

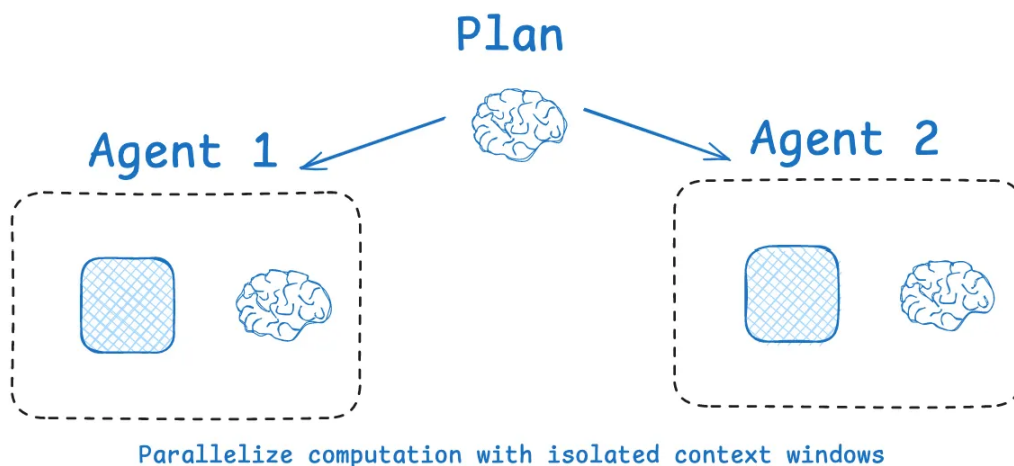


图9 通过多智能体拆分上下文

Anthropic 的多智能体研究助手也为这种做法提供了论据：很多具备独立上下文的子智能体组合起来，效果往往优于单个大智能体——主要原因是每个子智能体的上下文窗口都可以专注于一个更窄的子任务。正如其博客所说：

“[子智能体] 并行工作，各自拥有独立的上下文窗口，同时探索问题的不同方面。”

当然，多智能体方案也面临挑战：token 用量会显著上升（Anthropic 报告最多可达普通对话的15倍），规划子智能体工作需要精心的提示词工程，子智能体之间也需要协调。

## 6.2 用环境隔离上下文 (Sandbox / CodeAgent)

HuggingFace 的 deep researcher 展示了另一种有趣的上下文隔离方式。大多数智能体使用工具调用 API，返回 JSON 对象（工具参数）传给工具（例如搜索 API）再获取反馈（搜索结果）。HuggingFace 采用的是 CodeAgent：它输出包含目标工具调用的代码，这些代码在沙盒（sandbox）中运行，然后把从工具调用中筛选出的上下文（例如返回值）再回传给 LLM。

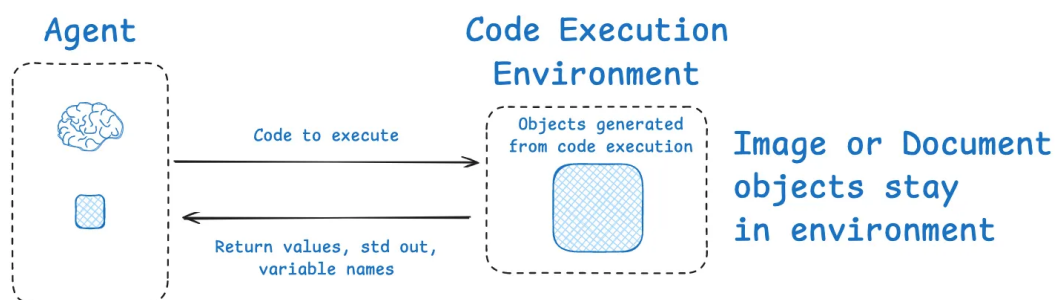


图10 沙盒可以把大体积对象与LLM隔离开



这种方式让上下文可以在“环境”里与 LLM 隔离开。HuggingFace 指出，这对于隔离高 token 体积的对象尤其有用：

“Code Agent 可以更好地处理状态……需要把这张图片/音频/其他东西存起来以后再用？没问题，在状态中把它赋给一个变量，之后直接使用就行。”

## 6.3 状态 (State)

值得一提的是，智能体的运行时状态对象本身也是隔离上下文的好办法，作用类似于沙盒。可以为状态设计一个 schema，让上下文写入到不同字段中：在智能体的每一轮，只有某个字段（例如 messages）暴露给 LLM，而 schema 中其他字段的信息则被隔离开，按需要选择性地使用。

# 七、用 LangSmith / LangGraph 做上下文工程

那么，如何把这些思路落到实践里？在动手之前，有两个基础设施非常关键。第一，要确保你有办法查看数据、跟踪智能体的使用情况——这能帮你判断把精力花在哪些上下文工程环节上收益最大。LangSmith 就非常适合做智能体 tracing / 可观测性，是这一步的好工具。第二，要确保你有一套简单的方法来测试上下文工程到底是帮了忙还是添了乱——LangSmith 提供了智能体评测能力，可以量化任何上下文工程改动对效果的影响。

## 7.1 写入上下文

LangGraph 在设计时同时支持了线程级短期记忆和长期记忆。短期记忆通过 checkpointing 在智能体各步骤之间持久化状态，非常适合作为“草稿本”，让你可以把信息写入状态，并在智能体轨迹的任意步骤读取。

LangGraph 的长期记忆则允许你在跨越多轮会话的场景下持久化上下文。它非常灵活：既可以保存少量文件（例如用户画像或规则），也可以保存大规模的记忆集合。此外，LangMem 还提供了一组丰富的抽象，辅助 LangGraph 的记忆管理。

## 7.2 筛选上下文

在 LangGraph 智能体的每一个节点（步骤）中，你都可以读取状态——这让你对每一步暴露给 LLM 的上下文拥有细粒度的控制权。

此外，LangGraph 的长期记忆可在每个节点内访问，支持多种检索方式（包括文件拉取、对记忆集合的嵌入检索等）。想了解长期记忆概览，可以看我们在 [Deeplearning.ai](#) 上的课程；想了解长期记忆在具体智能体中的落地入口，可以看 [Ambient Agents](#) 课程——它演示了如何在一个可以管理邮件、并从用户反馈中学习的长时运行智能体中使用 LangGraph 记忆。

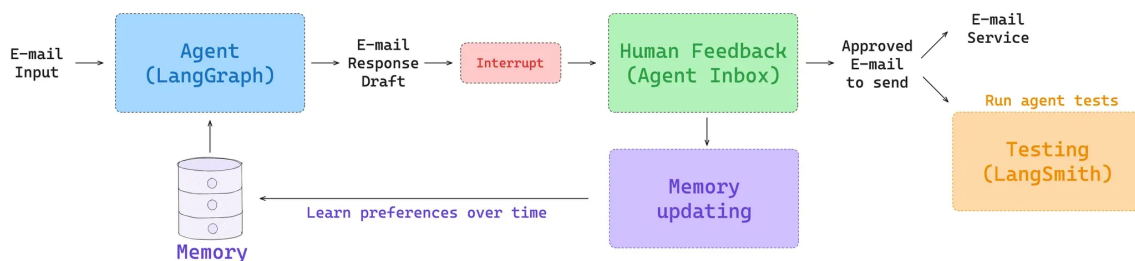


图11 一个使用用户反馈与长期记忆的邮件智能体

在工具筛选方面，LangGraph Bigtool 库是对工具描述做语义检索的好工具，在工具数量很大时，帮助筛选出当前任务最相关的工具。此外，我们还提供了大量教程和视频，演示如何在 LangGraph 中使用各类 RAG 方案。

## 7.3 压缩上下文

由于 LangGraph 是一个底层编排框架，你可以把智能体布局为一组节点，在每个节点内定义逻辑，并在节点之间传递状态对象。这种控制能力为压缩上下文提供了多种方式。

一个常见做法是：用消息列表作为智能体状态，并周期性地使用内置工具对其做摘要或修剪。但你也可以在不同位置添加逻辑：既可以在特定节点做摘要，也可以在工具调用节点中加入摘要逻辑，对特定工具的输出做压缩。

## 7.4 隔离上下文

LangGraph 围绕状态对象设计：你可以定义一个状态 schema，在每一步访问状态。例如，可以把工具调用返回的上下文存放在状态的某些字段中，把它们与 LLM 隔离，直到真正需要时再取出。除了状态隔离之外，LangGraph 也支持使用沙盒做上下文隔离——我们提供了一个使用 E2B 沙盒运行工具调用的 LangGraph 智能体示例仓库，也有视频演示了使用 Pyodide 做沙盒化、并持久化状态的方法。LangGraph 还对多智能体架构提供了丰富支持，例如 supervisor 和 swarm 模式的库，相关视频中也详细讲解了如何在 LangGraph 中使用多智能体。

# 八、结语

上下文工程正在成为智能体开发者必须掌握的一门手艺。本文梳理了当下主流智能体中常见的几类模式：

- 写入上下文 (Write)：把信息保存在上下文窗口之外，以帮助智能体完成任务；
- 筛选上下文 (Select)：把信息拉取到上下文窗口中，以帮助智能体完成任务；
- 压缩上下文 (Compress)：只保留完成任务所必需的 token；
- 隔离上下文 (Isolate)：把上下文拆分，以帮助智能体完成任务。

LangGraph 让你可以方便地实现上述每一种策略；LangSmith 则提供了测试智能体、跟踪上下文使用量的便捷工具。LangGraph + LangSmith 共同构成一个良性反馈循环：发现上下文工程的最佳机会 → 落地实现 → 评测效果 → 循环迭代。

— 完 —